

**GRUPO 1 [6 valores]**

Classifique as seguintes afirmações de verdadeiras (V) ou falsas (F). Uma resposta classificada corretamente conta 0,25 valores, incorrectamente desconta 0,25 valores ao total do grupo, sem resposta conta 0 valores.

1. [1] Segundo as regras definidas no protocolo HTTP, pode-se usar o método PUT na criação de um recurso:
  - ☐ Quando o recurso é único, ou seja, não é uma coleção de outros recursos.
  - ☐ O recurso a criar é uma coleção de recursos.
  - ☐ O URI do recurso é conhecido antes deste ser criado.
  - ☐ O URI do recurso a criar não tem componentes variáveis.
  
2. [1] Um método HTTP seguro significa que:
  - ☐ O mesmo pedido repetido várias vezes não altera o estado da aplicação.
  - ☐ Só utilizadores autenticados têm permissões para o executar.
  - ☐ Só aplicações autorizadas têm permissões para o executar.
  - ☐ Repetindo o mesmo pedido várias vezes, o resultado é sempre o mesmo.
  
3. [1] Na linguagem JavaScript, todos os objetos construídos através da mesma função construtora:
  - ☐ Têm sempre as propriedades criadas nessa função até ao final do programa.
  - ☐ Ao longo do programa podem vir a ter um número diferente de propriedades das criadas na função, mas todos os objetos têm sempre as mesmas.
  - ☐ Cada objeto pode vir a ter um número diferente de propriedades, mas há um conjunto destas que todos têm sempre.

Podem ter propriedades diferentes, assim que são retornados pela função construtora.
  
4. [1] A plataforma Node.js é um ambiente de execução:
  - ☐ Assíncrono e não bloqueante, o que significa que torna impossível bloquear o processo onde um programa corre.
  - ☐ Com uma única thread, o que significa que não suporta processamento paralelo.
  - ☐ Vocacionado para processamento assíncrono, mas que também suporta processamento síncrono.
  - ☐ Baseado num *loop* para a realização de operações não bloqueantes.
  
5. [1] O módulo Express:
  - ☐ Retorna um objeto que representa uma aplicação express.
  - ☐ Retorna um objeto que é um gerador de aplicações express.
  - ☐ É baseado em middlewares, que são funções cujo valor de retorno não é usado.
  - ☐ É baseado em middlewares, que são funções às quais são passados 3 parâmetros.
  
6. [1] No contexto do *web browser*:
  - ☐ A um documento HTML apenas pode ser aplicada uma CSS.
  - ☐ Este é uma aplicação que traduz dados em texto e conteúdos multimédia (texto, imagens, vídeo e sons).
  - ☐ Para que o browser apresente uma página web, tem que antes obter um documento HTML.
  - ☐ A mesma CSS pode ser aplicada a mais que um documento HTML.

## GRUPO 2 [10 valores]

7. [2] Pretende-se que o output do programa seguinte seja o que está nos comentários das linhas 3, 4 e 5. Implemente a função `Coll`, de modo a ter este comportamento.

```

1  const coll = new Coll([1,2,3,4,5])
2  coll.addItem(6); coll.addItem(7);
3  console.log(coll.removeItem()) // 7
4  console.log(coll.removeItem()) // 6
5  console.log(coll.removeItem()) // 5

```

8. [8] Considere o pedido HTTP e a correspondente resposta, na Listagem 1. O código que processou o pedido encontra-se na Listagem 2. Pretende-se que a resposta contenha o valor da chave 'a' presente no pedido.

| Listagem 1                                                                                                                                                                                      | Listagem 2                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> POST 3000/tasks HTTP/1.1 Accept: */* Host: localhost:3000 Content-Length: 7  a=value ----- HTTP/1.1 200 Date: Wed, 17 Jan 2018 23:14:41 GMT Content-Type: text/html  a = undefined </pre> | <pre> const express = require('express'); const bodyParser = require('body-parser'); const app = express(); app.use(bodyParser.urlencoded()); app.post('/tasks', function (req, rsp) {   rsp.end(`a = \${req.body.a}`); }); </pre> |

- a. [2] Identifique os problemas que encontra, quer provoquem erros funcionais ou não, e proponha as respetivas correções.
- b. [3] Implemente o módulo `restMw` que exporta uma função com a assinatura: `function(method, fn)`, que constrói um middleware express, com base nas funções passadas por parâmetro ao método `add` (**não pode usar o tipo Router do express**).

Assuma que as funções adicionadas (e.g. `foo`, `bar`, ou outras) recebem um único parâmetro, que é uma função que obedece à convenção de chamada com callback do node.js.

|                                                                                                                                      |                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> function foo(cb){ cb(null, 'I am foo')} function bar(cb){ cb(null, 'I am bar')} function zaz(cb){ cb(new Error('Bum'))} </pre> | <pre> const app = require('express')() const restMw = require('./restMw') const router = restMw('put', foo).add('get', zaz).add('get', bar) app.use(router) app.use((req, res) =&gt; res.status(404).send('Not Found')) app.use((err, req, res, next) =&gt; res.status(500).send(err.message)) app.listen(3000) </pre> |
|--------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Exemplos:

Pedido HTTP GET /bar tem uma resposta com *body* 'I am bar' e *status code* 200  
 Pedido HTTP GET /foo tem uma resposta com *body* 'Not Found' e *status code* 404  
 Pedido HTTP PUT /foo tem uma resposta com *body* 'I am foo' e *status code* 200  
 Pedido HTTP GET /xpto tem uma resposta com *body* 'Not Found' e *status code* 404  
 Pedido HTTP GET /zaz tem uma resposta com *body* 'Bum' e *status code* 500

O *path* para cada função corresponde ao nome da função e ao método HTTP especificado, não sendo portanto suportadas funções anónimas.

Exemplo: `...add('get', zaz)` adiciona uma rota para o path `'/zaz'` e o método HTTP GET.

Em caso de sucesso o resultado da função é retornado em **JSON** como resposta ao pedido HTTP.

Se um pedido HTTP não for respondido por nenhuma das funções do *middleware* construído por `restMw` então o atendimento é passado ao próximo *middleware* da aplicação express.

- c. [3] Implemente o módulo `memMw` que retorna um *middleware* express que memoriza a resposta dada aos pedidos HTTP que retornam conteúdos em JSON, como é o caso do *middleware* construído por `restMw`. Pedidos futuros para o mesmo *path* deverão retornar a resposta memorizada, sem que o pedido HTTP chegue ao *middleware* que o produziu num pedido anterior. O *middleware* dado `memMw` deve ser adicionado a uma aplicação Express antes de qualquer outro que produza respostas HTTP no formato JSON e que se queiram memorizar.

### GRUPO 3 [4 valores]

9. [4] Considere a aplicação TRINKAS desenvolvida no trabalho prático de PI.

- a. [1,5] Pretende-se adicionar a funcionalidade de associar várias notas a uma receita, independentemente do grupo onde esta se encontra. Cada nota tem um título e um texto associado. Não podem existir 2 notas com o mesmo título. Especifique o recurso a disponibilizar na API da componente servidora da aplicação para suportar esta funcionalidade. Na definição do recurso, indique o que for necessário para que quem usa a API tenha a informação para aceder a esse recurso e conseguir adicionar apenas uma nota a uma receita presente num grupo, bem como para saber identificar e interpretar as situações de erro.

NOTAS:

- O pedido tem o formato Json
- A resposta tem o formato Json, quer indique conclusão com sucesso ou não.
- Os erros suportados são: pedido inválido, recurso não encontrado e erro no servidor.

- b. [2,5] Implemente o método `addNoteToReceipe(receipeId, note)` do módulo `trinkas-services`, que recebe o identificador da receita em `receipeId`, uma nota em `note`, e um *callback* `cb`. Na sua implementação chama métodos dos módulos `trinkas-db` e `spoonacular-data`.

NOTA: Não implemente o métodos dos módulos `trinkas-db` e `spoonacular-data`, descreva apenas o que estes fazem.

O docente,  
Luís Falcão